



Instituto Tecnológico de Mazatlán

Inclinómetro con control de motorreductor

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Hernández Barrón

Alumnos:

Rosa Guadalupe Romero Navarro

Javier Ramón Quintero Valdez

Gabriel Alejandro Barrón Sarabia

Reporte de práctica: Inclínómetro con control de motorreductor

1. Descripción del proyecto

Esta práctica (3.2.3) consiste en construir un sistema embebido que mide la inclinación de un objeto con una unidad de medición inercial (IMU) MPU-6050 y utiliza esa información para controlar un motorreductor por medio de un puente H L298N, todo con un Arduino UNO R4 WiFi.

El sistema mide la inclinación frontal (pitch), es decir, el giro del sensor hacia adelante o hacia atrás sobre su eje Y. El signo del ángulo decide la dirección de giro del motor y su magnitud decide la velocidad: dentro de $\pm 5^\circ$ el motor permanece detenido, entre 5° y 45° la velocidad crece en proporción a la inclinación y a partir de 45° el motor trabaja a su velocidad máxima.

El programa está temporizado con `millis()` y `micros()`, sin utilizar `delay()` dentro del ciclo principal. Incluye una rampa de aceleración para que el motor responda de forma suave y un paro de seguridad que lo detiene de inmediato cuando falla la lectura del sensor, con recuperación automática. La inclinación se muestra en la matriz de LEDs de la placa y el estado del sistema en el Monitor Serie.

2. Objetivos

- Leer el sensor MPU-6050 por I2C accediendo directamente a sus registros, sin librerías externas.
- Calcular la inclinación frontal combinando el acelerómetro y el giroscopio con un filtro complementario.
- Controlar la dirección y la velocidad de un motorreductor con un puente H L298N y una señal PWM.
- Aplicar una zona muerta, una velocidad mínima útil y una rampa de aceleración para obtener una respuesta suave del motor.
- Implementar un paro de seguridad ante fallas del sensor, con recuperación automática.
- Mostrar la inclinación en la matriz de LEDs y el estado del sistema en el Monitor Serie.
- Organizar cuatro tareas periódicas con `millis()` sin bloquear el programa.

3. Herramientas y material utilizado

Hardware:

- Arduino UNO R4 WiFi (con matriz de LEDs 12x8 integrada)
- Módulo GY-521 con sensor MPU-6050 (acelerómetro y giroscopio, I2C)

- Módulo puente H L298N
- Motorreductor DC con llanta
- Fuente de alimentación externa para el motor
- Cables de conexión
- Cable USB-C

Software:

- Arduino IDE
- Librería Wire
- Librería Arduino_LED_Matrix
- Monitor Serie

4. Diagrama

El sensor MPU-6050 se conecta al bus I2C del Arduino con cuatro cables: VCC a 5V, GND a GND, SDA al pin SDA (A4) y SCL al pin SCL (A5). El puente H L298N recibe tres señales de control: ENA en el pin D9, que es una salida PWM y define la velocidad, e IN1 e IN2 en los pines D8 y D7, que definen la dirección de giro. Para que el Arduino pueda controlar la velocidad se retira el jumper de ENA del módulo.

El motor se conecta a las salidas OUT1 y OUT2 y se alimenta desde una fuente externa conectada a la terminal de +12V del L298N, nunca desde el pin de 5V del Arduino. La tierra de la fuente, la del L298N y la del Arduino se unen en una tierra común, que es indispensable para que las señales de control tengan la misma referencia.

El encabezado del programa recomienda además dos conexiones que no aparecen en el diagrama: el pin AD0 del sensor a GND, para fijar la dirección 0x68, y una resistencia de 10 kΩ entre ENA y GND, que mantiene el motor deshabilitado mientras el Arduino arranca o se reinicia.

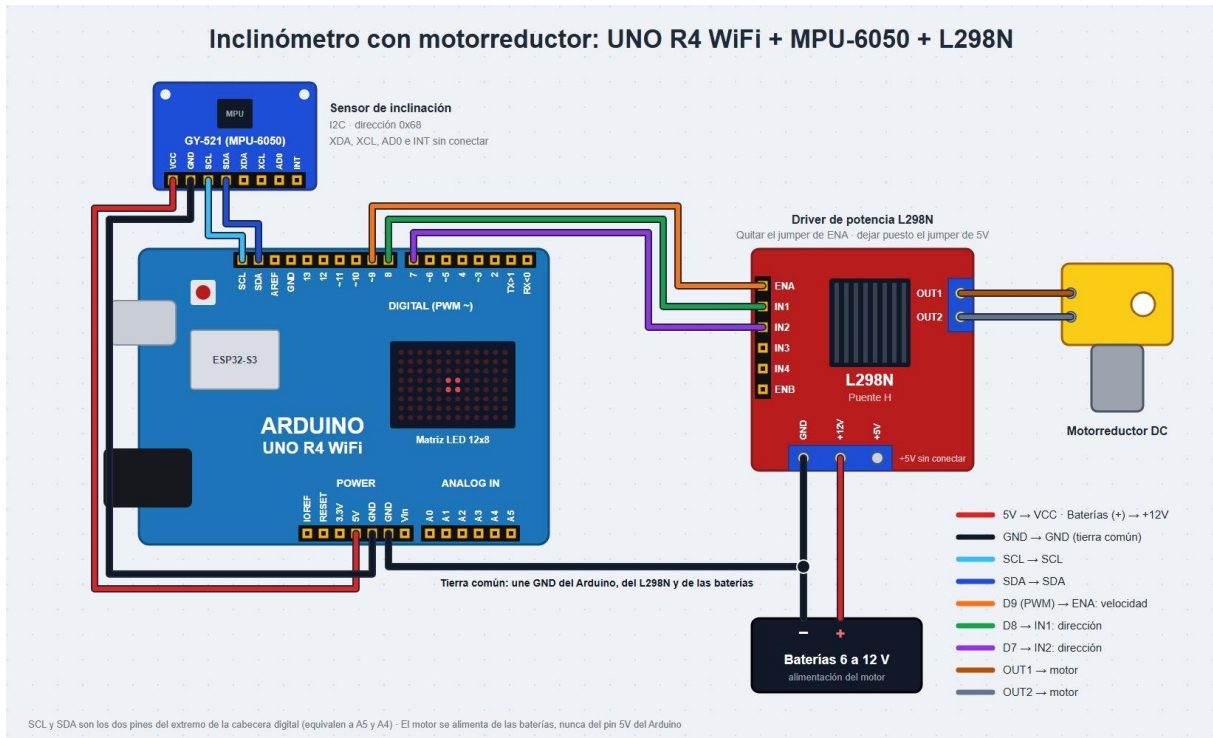


Diagrama de conexión del sensor, el puente H y el motorreductor con el Arduino UNO R4 WiFi.

5. Código

El siguiente código deja el motor apagado, verifica y configura el sensor, calibra el giroscopio y después ejecuta cuatro tareas periódicas: la lectura del sensor con el filtro cada 10 ms, la rampa del motor cada 20 ms, el Monitor Serie cada 500 ms y la matriz de LEDs cada 50 ms:

/*

Practica 3.2.3: Inclinómetro con control de motorreductor.
Arduino UNO R4 WiFi + MPU-6050 + modulo puente H L298N.

Motor: ENA=D9 (sin jumper ENA), IN1=D8, IN2=D7.
Sensor: bus Wire, SDA=A4, SCL=A5; AD0 a GND (direccion 0x68).
Consultar Guia_Inclinometro_R4.md antes de conectar la alimentacion.
Poner una resistencia de 10 kohm entre ENA y GND: mantiene el motor deshabilitado mientras el Arduino arranca o se reinicia.

Mantener el sensor QUIETO durante las 500 muestras de calibracion.
No utiliza WiFi ni necesita una biblioteca externa para el MPU-6050.

*/

```
#include <Arduino.h>
#include <Wire.h>
#include <Arduino_LED_Matrix.h>
#include <math.h>
#include <string.h>
```

```

const uint8_t PIN_ENA = 9;
const uint8_t PIN_IN1 = 8;
const uint8_t PIN_IN2 = 7;
const uint8_t DIRECCION_MPU = 0x68;

const float ZONA_MUERTA = 5.0f;
const float ZONA_CENTRO = 2.0f;
const float ANGULO_MAXIMO = 45.0f;
const int PWM_MINIMO = 90;
const int PWM_MAXIMO = 255;
const int PASO_RAMPA = 9; // 29 pasos x 20 ms = 580 ms.
const float TAU_FILTRO = 0.50f; // Alfa aproximado: 0.98 a 100 Hz.
const float SENTIDO_PITCH = 1.0f; // Cambiar a -1.0f si queda invertido.

const uint32_t PERIODO_SENSOR = 10;
const uint32_t PERIODO_MOTOR = 20;
const uint32_t PERIODO_SERIE = 500;
const uint32_t PERIODO_MATRIZ = 50;
const uint32_t PERIODO_REINTENTO = 250;
const uint32_t ESPERA_DESPERTAR = 100;
const unsigned int TIMEOUT_I2C_US = 5000;
const uint16_t MUESTRAS_CALIBRACION = 500;

struct LecturaIMU {
    float ax, ay, az; // Aceleracion en g.
    float gy; // Giro sobre Y, en grados/segundo.
};

enum EstadoSistema { BLOQUEADO, ACTIVO, PARO, RECUPERANDO };

// Prototipos explicitos: tambien permiten usar LecturaIMU en el IDE Arduino.
bool escribirRegistro(uint8_t registro, uint8_t valor);
bool leerRegistros(uint8_t registro, uint8_t *datos, size_t cantidad);
bool identificarIMU();
bool configurarIMU();
bool leerIMU(LecturaIMU &lectura);
float pitchAcelerometro(const LecturaIMU &lectura);
bool calibrarGiroscopio();
void iniciarBus();
void detenerMotor();
void aplicarMotor();
int calcularObjetivo(float angulo);
void actualizarRampa();
void activarParo(const char *motivo);
void actualizarSensor();
void intentarRecuperacion(uint32_t ahora);
void informarCambios();
void actualizarMatriz();

ArduinoLEDMatrix matriz;
EstadoSistema estado = BLOQUEADO;
float offsetGy = 0.0f;

```

```

float pitch = 0.0f;
int pwmObjetivo = 0; // El signo indica la direccion.
int pwmActual = 0; // PWM aplicado, NO velocidad medida.
uint32_t ultimoSensor = 0, ultimoMotor = 0, ultimoSerie = 0;
uint32_t ultimaMatriz = 0, ultimoReintento = 0, inicioRecuperacion = 0;
uint32_t ultimaLecturaUs = 0;
bool primerReporte = true;
int ultimoGrado = 0, ultimoPorcentaje = 0, ultimoSentidoMotor = 0;
int ultimoCentro = -1, ultimaIntensidad = -1, ultimoSentidoSensor = 0;

void iniciarBus() {
    Wire.begin();
    Wire.setClock(100000); // I2C a 100 kHz.
    Wire.setWireTimeout(TIMEOUT_I2C_US); // API del paquete UNO R4 / Renesas.
}

bool escribirRegistro(uint8_t registro, uint8_t valor) {
    Wire.beginTransaction(DIRECCION_MPU);
    Wire.write(registro);
    Wire.write(valor);
    return Wire.endTransmission(true) == 0;
}

bool leerRegistros(uint8_t registro, uint8_t *datos, size_t cantidad) {
    Wire.beginTransaction(DIRECCION_MPU);
    Wire.write(registro);
    if (Wire.endTransmission(false) != 0) return false;
    if (Wire.requestFrom(DIRECCION_MPU, cantidad, true) != cantidad) return false;
    for (size_t i = 0; i < cantidad; ++i) {
        if (!Wire.available()) return false;
        datos[i] = (uint8_t)Wire.read();
    }
    return true;
}

bool identificarIMU() {
    uint8_t identificacion = 0;
    return leerRegistros(0x75, &identificacion, 1) && identificacion == 0x68;
}

bool configurarIMU() {
    return escribirRegistro(0x6B, 0x01) && // Despertar, reloj PLL giroscopio X.
        escribirRegistro(0x6C, 0x00) && // Habilitar acelerometro y giroscopio.
        escribirRegistro(0x1B, 0x00) && // Giroscopio: +/-250 grados/s.
        escribirRegistro(0x1C, 0x00) && // Acelerometro: +/-2 g.
        escribirRegistro(0x1A, 0x03) && // Filtro interno: aproximadamente 44
Hz.
        escribirRegistro(0x19, 0x09); // 1000/(1+9) = 100 muestras/s.
}

bool leerIMU(LecturaIMU &lectura) {
    uint8_t datos[14];
    if (!leerRegistros(0x3B, datos, sizeof(datos))) return false;

```

```

    const int16_t ax = (int16_t)((uint16_t)datos[0] << 8 | datos[1]);
    const int16_t ay = (int16_t)((uint16_t)datos[2] << 8 | datos[3]);
    const int16_t az = (int16_t)((uint16_t)datos[4] << 8 | datos[5]);
    const int16_t gy = (int16_t)((uint16_t)datos[10] << 8 | datos[11]);
    lectura.ax = ax / 16384.0f;
    lectura.ay = ay / 16384.0f;
    lectura.az = az / 16384.0f;
    lectura.gy = gy / 131.0f;
    // Rechazar un vector nulo: no permite conocer la inclinacion.
    const float norma2 = lectura.ax * lectura.ax + lectura.ay * lectura.ay +
        lectura.az * lectura.az;
    return norma2 > 0.000001f;
}

float pitchAcelerometro(const LecturaIMU &lectura) {
    const float vertical = sqrtf(lectura.ay * lectura.ay + lectura.az *
    lectura.az);
    return SENTIDO_PITCH * atan2f(-lectura.ax, vertical) * 180.0f / PI;
}

bool calibrarGiroscopio() {
    float suma = 0.0f;
    LecturaIMU lectura;
    for (uint16_t i = 0; i < MUESTRAS_CALIBRACION; ++i) {
        if (!leerIMU(lectura)) return false;
        suma += lectura.gy;
        delay(PERIODO_SENSOR); // Solo en setup; motor apagado.
    }
    offsetGy = suma / MUESTRAS_CALIBRACION;
    return true;
}

void detenerMotor() {
    analogWrite(PIN_ENA, 0); // Quitar potencia primero.
    digitalWrite(PIN_IN1, LOW);
    digitalWrite(PIN_IN2, LOW);
    pwmActual = 0;
    pwmObjetivo = 0;
}

void aplicarMotor() {
    if (pwmActual == 0) {
        analogWrite(PIN_ENA, 0);
        digitalWrite(PIN_IN1, LOW);
        digitalWrite(PIN_IN2, LOW);
        return;
    }
    digitalWrite(PIN_IN1, pwmActual > 0 ? HIGH : LOW);
    digitalWrite(PIN_IN2, pwmActual > 0 ? LOW : HIGH);
    analogWrite(PIN_ENA, abs(pwmActual));
}

int calcularObjetivo(float angulo) {

```

```

const float magnitud = fabsf(angulo);
if (magnitud < ZONA_MUERTA) return 0;
const float limitado = magnitud > ANGULO_MAXIMO ? ANGULO_MAXIMO : magnitud;
const float proporcional = (limitado - ZONA_MUERTA) /
                           (ANGULO_MAXIMO - ZONA_MUERTA);
const int pwm = (int)(PWM_MINIMO + proporcional * (PWM_MAXIMO - PWM_MINIMO) +
0.5f);
return angulo >= 0.0f ? pwm : -pwm;
}

void actualizarRampa() {
    if (estado != ACTIVO) {
        detenerMotor();
        return;
    }
    // Al invertir: primero llegar a cero. La direccion cambia en otro paso.
    int destino = pwmObjetivo;
    if ((pwmActual > 0 && destino < 0) || (pwmActual < 0 && destino > 0)) destino =
0;
    if (pwmActual < destino) {
        pwmActual += PASO_RAMPA;
        if (pwmActual > destino) pwmActual = destino;
    } else if (pwmActual > destino) {
        pwmActual -= PASO_RAMPA;
        if (pwmActual < destino) pwmActual = destino;
    }
    aplicarMotor();
}

void activarParo(const char *motivo) {
    detenerMotor(); // Sin rampa en una falla.
    if (estado == ACTIVO) {
        Serial.print("PARO DE SEGURIDAD: ");
        Serial.println(motivo);
    }
    estado = PARO;
    ultimoReintento = millis();
}

void actualizarSensor() {
    LecturaIMU lectura;
    if (!leerIMU(lectura)) {
        activarParo("fallo de lectura del MPU-6050");
        return;
    }
    const uint32_t ahoraUs = micros();
    const float dt = (uint32_t)(ahoraUs - ultimaLecturaUs) / 1000000.0f;
    ultimaLecturaUs = ahoraUs;
    if (dt <= 0.0f || dt > 0.10f) {
        activarParo("intervalo de control demasiado largo");
        return;
    }
    const float anguloAccel = pitchAcelerometro(lectura);
    const float velocidadGy = SENTIDO_PITCH * (lectura.gy - offsetGy);

```

```

    const float alfa = TAU_FILTRO / (TAU_FILTRO + dt);
    pitch = alfa * (pitch + velocidadGy * dt) + (1.0f - alfa) * anguloAccel;
    pwmObjetivo = calcularObjetivo(pitch);
}

void intentarRecuperacion(uint32_t ahora) {
    if (estado == PARO && (uint32_t)(ahora - ultimoReintento) >= PERIODO_REINTENTO)
    {
        ultimoReintento = ahora;
        Wire.end();
        iniciarBus();
        if (identificarIMU() && configurarIMU()) {
            estado = RECUPERANDO;
            inicioRecuperacion = millis();
        }
    } else if (estado == RECUPERANDO &&
        (uint32_t)(ahora - inicioRecuperacion) >= ESPERA_DESPERTAR) {
        LecturaIMU lectura;
        if (!leerIMU(lectura)) {
            activarParo("sensor aun no disponible");
            return;
        }
        // Conservar la calibracion; reiniciar angulo y rampa, sin integrar la pausa.
        pitch = pitchAcelerometro(lectura);
        detenerMotor();
        ultimaLecturaUs = micros();
        ultimoSensor = ultimoMotor = millis();
        primerReporte = true;
        ultimoCentro = -1;
        estado = ACTIVO;
        Serial.println("SENSOR RECUPERADO: control habilitado nuevamente.");
    }
}

void informarCambios() {
    if (estado != ACTIVO || !Serial) return;
    const float magnitud = fabsf(pitch);
    const int centro = magnitud <= ZONA_CENTRO ? 1 : 0;
    const int grado = (int)roundf(pitch);
    const int porcentaje = (abs(pwmActual) * 100 + PWM_MAXIMO / 2) / PWM_MAXIMO;
    const int sentidoMotor = (pwmActual > 0) - (pwmActual < 0);
    const int sentidoSensor = centro ? 0 : (pitch > 0.0f ? 1 : -1);
    const int intensidad = magnitud < 15.0f ? 0 : (magnitud < 30.0f ? 1 : 2);
    if (!primerReporte && grado == ultimoGrado && porcentaje == ultimoPorcentaje &&
        sentidoMotor == ultimoSentidoMotor && centro == ultimoCentro &&
        intensidad == ultimaIntensidad && sentidoSensor == ultimoSentidoSensor)
    return;

    if (centro != ultimoCentro) {
        if (centro) Serial.println("ENTRA AL CENTRO: sensor dentro de +/-2 grados.");
        else if (ultimoCentro == 1) Serial.println("SALE DEL CENTRO.");
    }
    Serial.print("Inclinacion: ");
    Serial.print(centro ? "centrado" : (sentidoSensor > 0 ? "adelante" : "atras"));
}

```

```

    Serial.print(" | ");
    Serial.print(grado);
    Serial.print(" grados | ");
    Serial.print(intensidad == 0 ? "leve" : (intensidad == 1 ? "moderada" :
"fuerte"));
    Serial.print(" | Motor: ");
    Serial.print(sentidoMotor == 0 ? "detenido" : (sentidoMotor > 0 ? "adelante" :
"reversa"));
    Serial.print(" | PWM: ");
    Serial.print(porcentaje);
    Serial.println(" %");

    ultimoGrado = grado;
    ultimoPorcentaje = porcentaje;
    ultimoSentidoMotor = sentidoMotor;
    ultimoCentro = centro;
    ultimaIntensidad = intensidad;
    ultimoSentidoSensor = sentidoSensor;
    primerReporte = false;
}

void actualizarMatriz() {
    uint8_t imagen[8][12] = {};
    if (estado != ACTIVO) {
        for (int fila = 0; fila < 8; ++fila) { // X: paro o fallo inicial.
            imagen[fila][fila + 2] = 1;
            imagen[fila][9 - fila] = 1;
        }
    } else if (fabsf(pitch) <= ZONA_CENTRO) {
        for (int fila = 0; fila < 8; ++fila)
            for (int columna = 0; columna < 12; ++columna)
                if (fila == 0 || fila == 7 || columna == 0 || columna == 11)
                    imagen[fila][columna] = 1;
    } else {
        const float limitado = constrain(pitch, -ANGULO_MAXIMO, ANGULO_MAXIMO);
        const int fila = (int)(3.0f - limitado * 3.0f / ANGULO_MAXIMO + 0.5f);
        imagen[fila][5] = imagen[fila][6] = 1;
        imagen[fila + 1][5] = imagen[fila + 1][6] = 1;
    }
    matriz.renderBitmap(imagen, 8, 12);
}

void setup() {
    // PRIMERA ACCION: fijar las salidas de motor a nivel bajo.
    digitalWrite(PIN_ENA, LOW);
    digitalWrite(PIN_IN1, LOW);
    digitalWrite(PIN_IN2, LOW);
    pinMode(PIN_ENA, OUTPUT);
    pinMode(PIN_IN1, OUTPUT);
    pinMode(PIN_IN2, OUTPUT);
    analogWriteResolution(8);
    detenerMotor();

    Serial.begin(115200);

```

```

const uint32_t inicioSerie = millis();
while (!Serial && (uint32_t)(millis() - inicioSerie) < 1500) delay(1);
matriz.begin();
iniciarBus();

if (!identificarIMU() || !configurarIMU()) {
    Serial.println("ERROR INICIAL: MPU-6050 no responde 0x68 o no se
configura.");
    Serial.println("Motor apagado. Revisar conexiones y pulsar RESET.");
    actualizarMatriz();
    return; // BLOQUEADO: no arranca con sensor malo.
}
delay(ESPERA_DESPERTAR);
Serial.println("MPU-6050 detectado. Mantener sensor QUIETO durante 5
segundos.");
if (!calibrarGiroscopio()) {
    Serial.println("ERROR INICIAL: fallo durante la calibracion. Pulsar RESET.");
    actualizarMatriz();
    return;
}
LecturaIMU lectura;
if (!leerIMU(lectura)) {
    Serial.println("ERROR INICIAL: no se pudo obtener el angulo inicial.");
    actualizarMatriz();
    return;
}
pitch = pitchAcelerometro(lectura);
ultimaLecturaUs = micros();
ultimoSensor = ultimoMotor = ultimoSerie = ultimaMatriz = millis();
estado = ACTIVO;
Serial.print("Calibracion lista. Offset Y: ");
Serial.print(offsetGy, 4);
Serial.println(" grados/s. Sistema listo.");
actualizarMatriz();
}

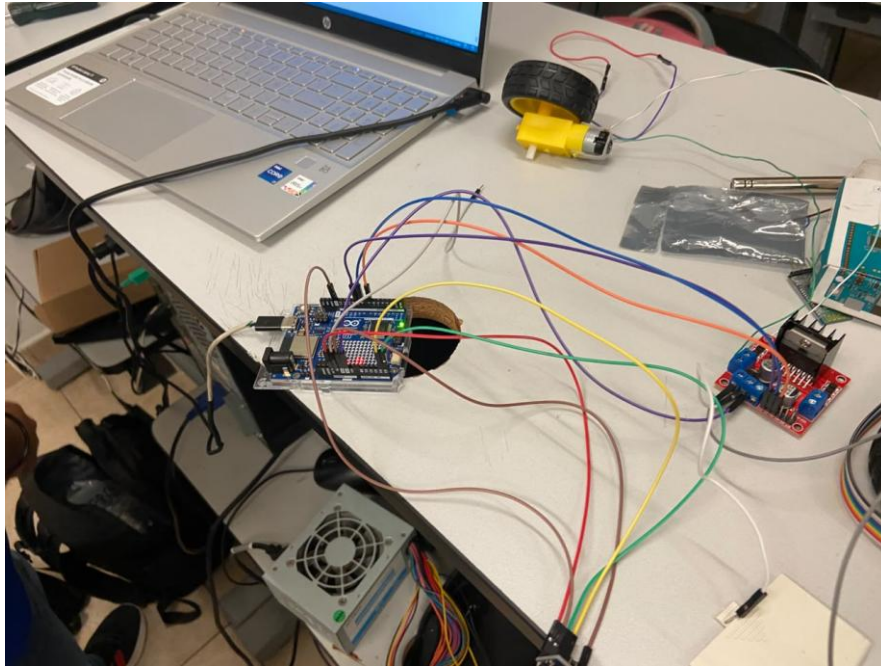
void loop() {
    const uint32_t ahora = millis();
    if (estado == ACTIVO && (uint32_t)(ahora - ultimoSensor) >= PERIODO_SENSOR) {
        ultimoSensor = ahora;
        actualizarSensor();
    }
    if (estado == PARO || estado == RECUPERANDO) intentarRecuperacion(millis());
    const uint32_t despues = millis();
    if ((uint32_t)(despues - ultimoMotor) >= PERIODO_MOTOR) {
        ultimoMotor = despues;
        actualizarRampa();
    }
    if ((uint32_t)(despues - ultimoSerie) >= PERIODO_SERIE) {
        ultimoSerie = despues;
        informarCambios();
    }
    if ((uint32_t)(despues - ultimaMatriz) >= PERIODO_MATRIZ) {
        ultimaMatriz = despues;
    }
}

```

```
    actualizarMatriz();  
  }  
  // Sin delay(): las cuatro tareas se programan con millis()/micros().  
}
```

6. Montaje físico

Como parte de la práctica se realizó el armado físico del circuito, replicando las conexiones del diagrama: el Arduino UNO R4 WiFi conectado por USB a la computadora, el módulo L298N con sus señales de control y el motorreductor con su llanta conectado a las salidas del puente H.



Montaje físico del circuito con el Arduino, el puente H L298N y el motorreductor.

7. Metodología

Primero se conectó el módulo GY-521 al bus I2C del Arduino UNO R4 WiFi y después el módulo L298N: se retiró el jumper de ENA, se unieron las señales ENA, IN1 e IN2 a los pines D9, D8 y D7, se conectó el motorreductor a las salidas OUT1 y OUT2 y se unieron las tierras del Arduino, del puente H y de la fuente del motor.

En el programa se definió como primera acción del setup() poner en nivel bajo las tres salidas del motor, de modo que nunca arranque solo al energizar el sistema. A continuación se implementó la verificación del sensor leyendo el registro WHO_AM_I (0x75), que debe responder 0x68; si no lo hace, el sistema queda bloqueado con el motor apagado.

Con el sensor identificado se configuraron sus rangos (± 2 g para el acelerómetro y ± 250 °/s para el giroscopio), su filtro interno y una frecuencia de 100 muestras por segundo. Después se calibró el giroscopio promediando 500 lecturas con el sensor quieto, lo que

tarda unos 5 segundos, y se calculó el ángulo inicial con el acelerómetro para que el filtro no arranque desde cero.

Dentro del loop() se programaron cuatro tareas independientes controladas con millis(): sensor y filtro cada 10 ms, rampa y motor cada 20 ms, Monitor Serie cada 500 ms y matriz de LEDs cada 50 ms. También se programó el paro de seguridad y el reintento de conexión con el sensor cada 250 ms.

Finalmente se cargó el programa, se abrió el Monitor Serie a 115200 baudios y se inclinó el sensor hacia adelante y hacia atrás para observar el ángulo, la dirección y el porcentaje de PWM del motor, así como el indicador en la matriz de LEDs.

8. Explicación del funcionamiento

El MPU-6050 contiene un acelerómetro y un giroscopio de tres ejes y funciona como esclavo del bus I2C en la dirección 0x68. El programa no utiliza una librería para el sensor: escribe y lee directamente sus registros. Los 14 bytes de datos se leen en una sola transacción a partir del registro 0x3B, y cada valor se arma con dos bytes, primero el alto y después el bajo. Las lecturas se convierten a unidades físicas dividiendo entre 16384 para obtener g y entre 131 para obtener grados por segundo.

El ángulo frontal se puede calcular de dos maneras. Con el acelerómetro se obtiene a partir de la dirección de la gravedad, con la expresión $\text{atan2}(-a_x, \sqrt{a_y^2 + a_z^2})$; es exacto en reposo, pero ruidoso cuando el sensor se mueve o vibra. Con el giroscopio se obtiene sumando la velocidad de giro multiplicada por el tiempo transcurrido; es suave, pero acumula error con el tiempo. El filtro complementario combina ambos: $\text{ángulo} = \alpha \cdot (\text{ángulo} + \text{giro} \cdot dt) + (1 - \alpha) \cdot \text{ángulo del acelerómetro}$. En el programa α se calcula como $\tau / (\tau + dt)$ con $\tau = 0.5$ s, lo que da aproximadamente 0.98 a 100 muestras por segundo. El offset del giroscopio obtenido en la calibración se resta de cada lectura.

Con el ángulo filtrado, la función calcularObjetivo() decide el movimiento. Si la magnitud del ángulo es menor a 5° devuelve 0 (zona muerta). Entre 5° y 45° calcula un PWM proporcional que va de 90 a 255, y a partir de 45° entrega 255. El valor lleva signo: positivo para adelante y negativo para reversa. El mínimo de 90 existe porque con un PWM menor el motor no logra vencer la fricción de sus engranes.

El puente H L298N permite invertir la polaridad del motor. Con IN1 en alto e IN2 en bajo el motor gira hacia adelante, con IN1 en bajo e IN2 en alto gira en reversa, y con ambos en bajo se detiene. La velocidad se controla con la señal PWM del pin ENA: mientras mayor es el valor, de 0 a 255, más tiempo permanece encendida la alimentación del motor en cada ciclo y más rápido gira.

El motor no salta directamente a la velocidad calculada. Cada 20 ms la función actualizarRampa() acerca el PWM aplicado al PWM objetivo en pasos de 9 unidades, por lo que pasar de detenido a velocidad máxima toma 29 pasos, unos 0.58 segundos. Cuando

el objetivo tiene el sentido contrario al actual, el destino intermedio es cero: el motor primero desacelera hasta detenerse y después acelera en el otro sentido.

El programa trabaja con cuatro estados: BLOQUEADO, ACTIVO, PARO y RECUPERANDO. Si en el arranque el sensor no responde o falla la calibración, el sistema queda en BLOQUEADO con el motor apagado. En operación normal está en ACTIVO. Si falla una lectura del sensor, o si el intervalo entre dos lecturas supera 0.1 segundos, se ejecuta activarParo(): el motor se detiene de inmediato, sin rampa, y se informa PARO DE SEGURIDAD en el Monitor Serie. En ese estado el programa reinicia el bus I2C cada 250 ms e intenta identificar y configurar de nuevo el sensor; cuando lo consigue espera 100 ms, toma el ángulo del acelerómetro como nuevo punto de partida y vuelve al estado ACTIVO conservando la calibración. El bus I2C tiene además un tiempo límite de 5 ms para que una falla de cableado no deje detenido al programa.

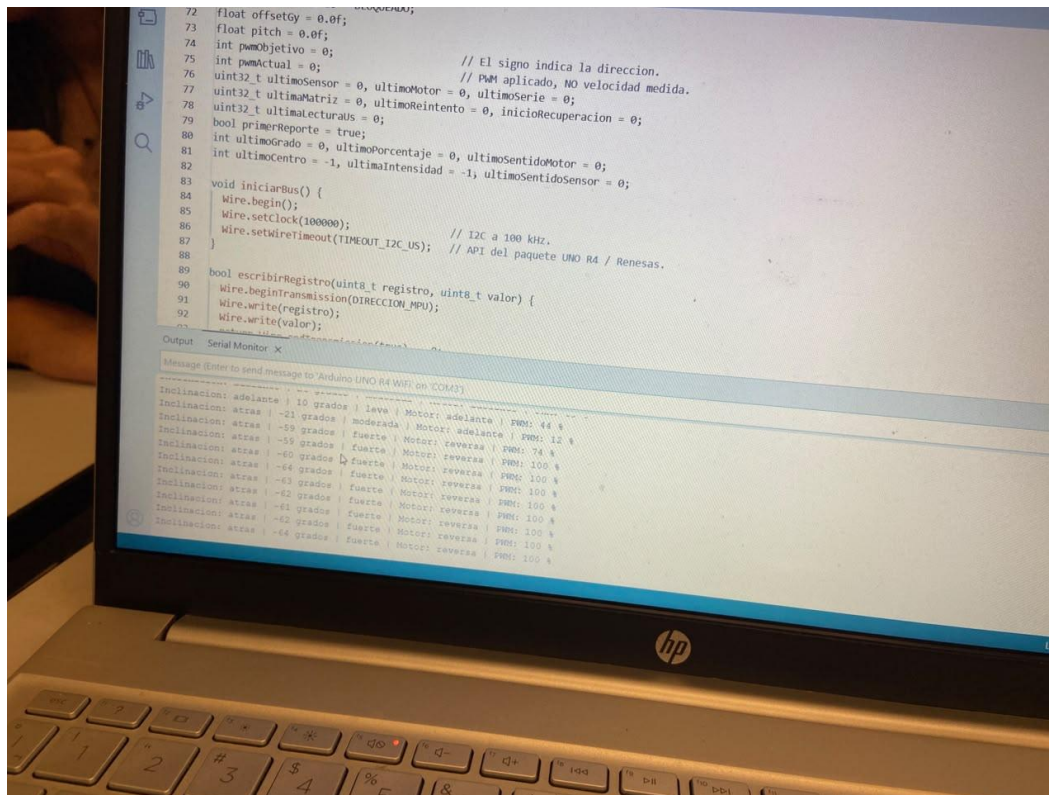
La matriz de LEDs funciona como indicador. Cuando la inclinación está dentro de $\pm 2^\circ$ se enciende el marco completo. Fuera de ese rango se dibuja un punto de 2×2 LEDs que se desplaza hacia arriba o hacia abajo según el ángulo, hasta llegar al borde a los 45° . Cuando el sistema no está activo, por un paro o por una falla inicial, se dibuja una X.

El Monitor Serie se revisa cada 500 ms, pero solo se imprime una línea cuando cambia el grado, el porcentaje de PWM, el sentido del motor o la intensidad, que se clasifica como leve (menos de 15°), moderada (menos de 30°) o fuerte. La entrada y la salida del centro se anuncian una sola vez.

Como ninguna tarea utiliza delay(), en cada vuelta del loop() se compara el tiempo actual con el de la última ejecución de cada tarea y solo se ejecuta la que ya cumplió su periodo. Así el sensor, el motor, el Monitor Serie y la matriz trabajan con periodos distintos sin interferir entre sí.

9. Monitor Serie

Con el sistema funcionando, el Monitor Serie muestra una línea cada vez que cambia la inclinación o el estado del motor. Cada línea indica la dirección de la inclinación, los grados, la intensidad, el sentido de giro del motor y el porcentaje de PWM aplicado:



Monitor Serie mostrando la inclinación y el estado del motor.

10. Análisis de resultados

Durante las pruebas el Arduino se comunicó correctamente con el sensor y el motor respondió a la inclinación tanto en dirección como en velocidad, con el formato de salida esperado en el Monitor Serie.

En la captura, con una inclinación de 10° hacia adelante, clasificada como leve, el motor giró hacia adelante con un PWM de 44 %. Este valor coincide con el cálculo del programa: $(10 - 5) / (45 - 5) = 0.125$, y $90 + 0.125 \times 165 \approx 111$, que equivale al 44 % de 255.

Al inclinar el sensor hacia atrás se observa el efecto de la rampa. En la lectura de -21° , el sensor ya estaba inclinado hacia atrás pero el motor seguía girando hacia adelante con solo 12 % de PWM, es decir, estaba desacelerando antes de invertir el giro. En la lectura siguiente, con -59° , el motor ya giraba en reversa al 74 %, y en las lecturas posteriores alcanzó el 100 %.

Mientras la inclinación se mantuvo entre -59° y -64° , clasificada como fuerte, el PWM permaneció en 100 %. Esto confirma que a partir de 45° la velocidad ya no aumenta, sin importar cuánto más se incline el sensor.

En el montaje físico la matriz de LEDs de la placa mostró el indicador de inclinación mientras el Monitor Serie seguía recibiendo datos, lo que muestra que las tareas se ejecutan de forma independiente con `millis()`, sin que una detenga a la otra.

11. Preguntas de análisis

1. ¿Por qué no basta con usar solo el acelerómetro para medir la inclinación? Explica qué observaste al sacudir el sensor.

El acelerómetro calcula el ángulo a partir de la dirección de la gravedad, pero no distingue la gravedad de cualquier otra aceleración. En reposo es exacto, pero al sacudir o vibrar el sensor mide también ese movimiento y lo interpreta como inclinación. Al sacudirlo sin inclinarlo, el ángulo calculado solo con el acelerómetro dio saltos bruscos de varios grados, mientras que el ángulo filtrado casi no se movió. Si el motor se controlara solo con el acelerómetro, cada vibración, incluidas las del propio motor, lo haría arrancar o cambiar de velocidad.

2. ¿Qué ocurriría si se usara solo el giroscopio durante varios minutos?

El giroscopio no mide ángulo, sino velocidad de giro, y el ángulo se obtiene sumando esas lecturas en el tiempo. Cada lectura trae un pequeño error que también se suma, por lo que el ángulo se va desviando poco a poco (deriva). Después de varios minutos marcaría varios grados de inclinación con el sensor quieto y nivelado, y el motor empezaría a girar solo. La calibración inicial reduce ese error, pero no lo elimina; por eso el filtro complementario usa el giroscopio para la respuesta rápida y el acelerómetro para corregir la deriva.

3. ¿Qué función cumple la zona muerta de $\pm 5^\circ$? ¿Qué pasaría si fuera de 0° ?

Define un rango alrededor de la posición nivelada en el que el motor permanece detenido, para que ignore el ruido residual del sensor, el pulso de la mano y las inclinaciones mínimas. Con una zona de 0° el motor nunca se detendría: el ángulo casi nunca es exactamente cero, así que estaría cambiando constantemente entre adelante y reversa con cualquier variación de décimas de grado, lo que provocaría vibración, calentamiento, desgaste y consumo inútil de energía.

4. ¿Por qué la velocidad mínima del motor es 90 y no 0?

Porque con un PWM bajo el motor recibe muy poca energía y no logra vencer la fricción de los engranes de la caja reductora: zumba y consume corriente, pero no gira. Si la velocidad empezara en 0, habría un tramo de inclinación justo después de la zona muerta en el que el sistema ordena moverse y no ocurre nada. Al empezar en 90, cerca del 35 %, el motor ya gira en cuanto sale de la zona muerta.

5. Explica dos problemas que evita la rampa de aceleración.

El primero son los picos de corriente: un arranque o una inversión de giro de golpe exige por un instante una corriente muy alta, que puede bajar el voltaje, reiniciar el microcontrolador o dañar las baterías y el driver. El segundo es el esfuerzo mecánico: un cambio brusco, sobre todo de adelante a reversa, golpea los dientes de los engranes y los

desgasta. Con la rampa la velocidad cambia en pasos pequeños cada 20 ms y, al invertir la dirección, el motor primero frena hasta cero y después acelera hacia el otro lado.

6. ¿Por qué el paro de seguridad se aplica de inmediato y no con rampa?

Porque cuando el sensor falla el sistema ya no sabe cuál es la inclinación real, y la rampa mantendría el motor girando alrededor de medio segundo más con base en un dato que ya no es confiable. En un sistema de control, si no se sabe qué está pasando, lo seguro es detener el actuador cuanto antes. La rampa protege al motor en la operación normal; en una falla la prioridad es la seguridad.

7. ¿Por qué el programa no usa delay()? ¿Qué pasaría con la respuesta del motor si se usara?

Porque delay() detiene todo el programa, y este sistema tiene cuatro tareas con periodos distintos: sensor cada 10 ms, motor cada 20 ms, matriz cada 50 ms y Monitor Serie cada 500 ms. Con millis() cada una se ejecuta cuando le corresponde sin bloquear a las demás. Si se usara, por ejemplo, un delay(500) para el Monitor Serie, el sensor se leería 2 veces por segundo en lugar de 100 y el filtro calcularía mal el ángulo; la rampa avanzaría un paso cada medio segundo, por lo que el motor tardaría unos 15 segundos en llegar a su velocidad máxima; y el paro de seguridad podría tardar hasta medio segundo en actuar. El motor respondería tarde y a saltos.

8. Menciona tres aplicaciones reales de este sistema.

- Vehículos autobalanceados, como las patinetas eléctricas y los Segway, donde la inclinación del usuario decide hacia dónde y qué tan rápido avanzan.
- Estabilizadores de cámara (gimbals) y plataformas niveladoras, en los que los motores compensan la inclinación para mantener nivelada la cámara o la superficie.
- Controles por inclinación, como mandos de videojuegos, controles de drones o sillas de ruedas que se manejan inclinando un control.

12. Conclusiones

Gabriel

En esta práctica comprendí que ninguno de los dos sensores del MPU-6050 sirve por sí solo para medir la inclinación: el acelerómetro se altera con cualquier vibración y el giroscopio se desvía con el tiempo. El filtro complementario me pareció una solución sencilla, de una sola línea, para quedarse con lo mejor de cada uno.

Javier

Aprendí que controlar un motor no es solo enviarle una velocidad. La zona muerta, la velocidad mínima y la rampa resuelven problemas reales, como que el motor vibre con el

ruido del sensor, que no arranque por la fricción o que sufra al cambiar de sentido de golpe. También reforcé el uso del puente H y de la señal PWM.

Rosa

Esta práctica me permitió entender la importancia del paro de seguridad: si el sistema deja de recibir información del sensor, lo correcto es detener el motor de inmediato y no seguir con el último dato. Además comprendí que usar `millis()` en lugar de `delay()` es lo que permite que el sistema reaccione a tiempo, porque ninguna tarea detiene a las demás.